When Scheduling Is Not Enough: Resource Governance for Heterogeneous LLM Inference

YieldOS-Lite Research Draft

Draft generated from MVP simulator results

Code and artifacts: https://github.com/nikitph/yieldos

Abstract

Modern large language model (LLM) serving systems improve throughput through iteration-level scheduling, continuous batching, KV-cache paging, chunked prefill, and prefill/decode disaggregation. These mechanisms optimize local execution, but mixed inference workloads expose a higher-level coordination problem: heterogeneous requests impose different obligations on prefill compute, decode bandwidth, KV-cache residency, latency slack, tenant priority, and prefix reuse. The objective of this paper is to test whether this coordination problem is worth treating as a distinct *resource-governance* layer before building a production serving-engine integration.

To answer that question, we introduce *YieldOS-Lite*, a dependency-free, trace-driven Phase 1 simulator for LLM inference resource governance. YieldOS-Lite evaluates a slow-path control plane that publishes policy snapshots consumed by a simple hot path. The current implementation includes predictive SLO governance (the SLO Notary), coarse value-aware KV accounting (the KV Treasury), advisory shape forecasting, governed-goodput metrics, decision logs, baseline policies, ablations, workload profiles, and replay compatibility. The central metric is *governed goodput*: completed output tokens from requests that satisfy both TTFT and ITL SLOs, divided by simulated GPU time plus control-plane overhead.

The simulator evidence supports a constrained claim. In synthetic heterogeneous workloads, YieldOS-Lite improves governed goodput over a DistServe-style disaggregated baseline by 26.5% in the default comparison and by 22.7–42.5% across load regimes from 50% to 150%. Semi-realistic profile runs show workload-dependent gains: +8.6% for chat-heavy traffic, +47.6% for RAG-heavy traffic, +36.5% for code-heavy traffic, +67.9% for batch-summary-heavy traffic, and +121.8% for mixed-enterprise traffic. An external BurstGPT pilot provides a boundary condition: in a homogeneous interactive sample with little prefix reuse, DistServe-style disaggregation is competitive or better. To explain this, we introduce a coarse *Obligation Heterogeneity Index* (OHI), which has a preliminary Pearson correlation of 0.713 with YieldOS gain across the pilot set. The takeaway is not that YieldOS is production-ready or universally better than existing engines. The takeaway is that resource governance is a promising research direction, with a runnable simulator today, and that its advantage appears when request streams contain heterogeneous obligations that phase separation alone cannot resolve.

1 Introduction

LLM inference serving has rapidly evolved from static request-level batching to iteration-level scheduling, continuous batching, KV-cache paging, chunked prefill, and disaggregated prefill/decode execution. Orca introduced iteration-level scheduling and selective batching for transformer serving [1]; vLLM popularized PagedAttention and high-throughput memory-efficient serving [2, 3]; Sarathi-Serve introduced chunked prefill and stall-free scheduling to improve throughput under tail-latency constraints [4]; DistServe disaggregates prefill and decode computation to reduce phase interference and improve goodput [5]. These systems are strong mechanistic schedulers.

The thesis of this paper is that the next bottleneck is not only mechanistic scheduling. It is *resource governance*. In homogeneous request streams, a small set of optimized scheduling rules may be sufficient.

In heterogeneous request streams, however, requests create competing claims on multiple scarce resources: GPU compute during prefill, memory bandwidth during decode, KV-cache residency, prefix reuse, tenant priority, latency slack, and scheduler attention. A serving system must decide not simply which request goes next, but which obligation deserves scarce resources now.

YieldOS-Lite is a Phase 1 simulator for this question. It does not replace CUDA kernels, vLLM, TensorRT-LLM, or production serving engines. It tests whether a slow-path resource-governance control plane can improve allocation behavior before integration with real engines. The current evidence supports a disciplined claim:

> YieldOS-Lite is not a better queue. It is a better response to heterogeneity.

1.1 Contributions

This paper makes six contributions.

1. It introduces **governed goodput**: SLO-valid completed output tokens per simulated GPU-second, net of control-plane overhead.
2. It introduces **YieldOS-Lite**, a dependency-free trace-driven simulator for evaluating resource-governance policies over LLM inference baselines.
3. It implements and evaluates **predictive SLO governance**, **value-aware KV accounting**, **advisory shape forecasting**, and **policy-cadence control** against continuous batching, Sarathi-style chunked prefill, and DistServe-style disaggregation.
4. It shows that YieldOS-Lite improves governed goodput over a DistServe-style baseline across synthetic heterogeneous load regimes and semi-realistic workload profiles.
5. It reports a boundary condition from a BurstGPT pilot: homogeneous interactive traces with little prefix reuse may favor mechanistic disaggregation.
6. It proposes an **Obligation Heterogeneity Index** (OHI) and finds a preliminary correlation of 0.713 between OHI and YieldOS gain across the pilot workload set.

2 Background and Motivation

LLM inference has two main phases. *Prefill* processes the input prompt and is compute-intensive; *decode* generates one output token at a time and often exhibits lower compute utilization but higher sensitivity to memory bandwidth and batching dynamics. Sarathi-Serve states this contrast explicitly and introduces chunked prefill to co-schedule prefills and decodes without stalling ongoing decodes [4]. DistServe observes that colocating prefill and decode causes strong interference and couples their resource allocation and parallelism decisions [5]. These observations motivate a systems-level separation of concerns.

YieldOS-Lite extends this direction. It assumes that existing mechanisms are necessary but not sufficient. The question is not whether to use continuous batching, chunked prefill, or disaggregation. The question is when to allocate scarce compute, memory, and latency slack to each class of obligation.

2.1 From Request Scheduling to Obligation Governance

Existing schedulers often treat a request as the basic scheduling unit, or split it into prefill/decode phases. YieldOS-Lite treats each request as a bundle of obligations:

- prefill compute obligations,

- decode bandwidth obligations,
- KV-residency obligations,
- SLO-deadline obligations,
- tenant-priority obligations,
- prefix-sharing obligations.

This reframing leads to a different design question:

> Which resource obligation, if served now, maximizes SLO-valid useful output under current contention?

3 System Overview

Figure 1 shows the YieldOS-Lite architecture. YieldOS-Lite uses a slow-path control plane that computes policy snapshots. The hot path consumes the current snapshot and remains deliberately simple.

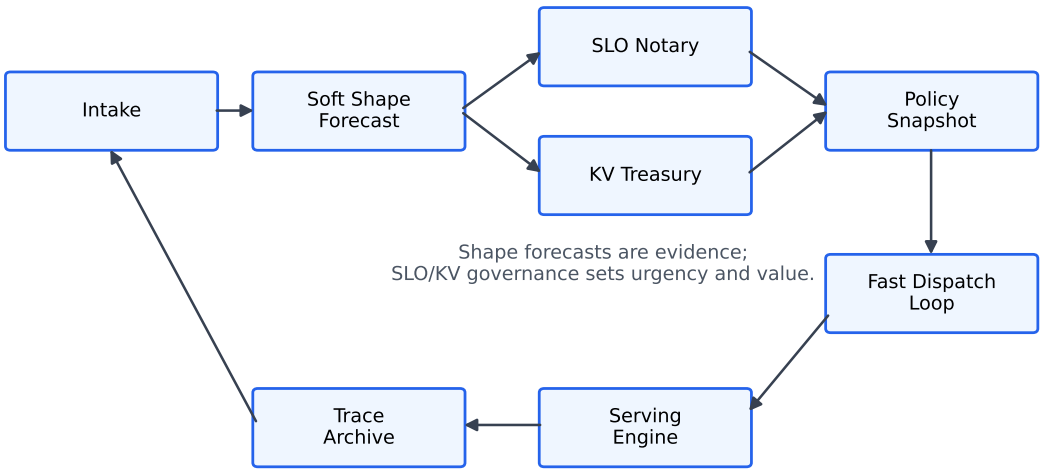

Figure 1: YieldOS-Lite architecture. Shape forecasts are advisory, not hard routing authority. The SLO Notary and KV Treasury produce slow-path policy snapshots consumed by the fast dispatch loop.

3.1 SLO Notary

The SLO Notary estimates breach risk before a request violates TTFT or ITL targets. It intervenes by raising urgency, protecting decode capacity, delaying conflicting work, or shaping admissions. The important distinction is that SLOs are not merely post-hoc metrics; they become live scheduling signals.

3.2 KV Treasury

The KV Treasury assigns coarse value to KV residency. Raw KV hits are not enough: preserving many low-value pages can be worse than preserving fewer high-value pages. The MVP value proxy is:

$$\text{KVValuePreserved} = T_{saved} \cdot P_{slo} \cdot P_{tenant} \cdot U_{future} \cdot S_{sharing} \tag{1}$$

where T_{saved} denotes saved prefix tokens, P_{slo} SLO pressure, P_{tenant} tenant priority, U_{future} expected future use, and $S_{sharing}$ sharing potential.

3.3 Shape Forecasting

The simulator includes probabilistic shape classification, but the current policy demotes it from hard routing authority to advisory evidence. The shape-stress result showed that hard routing fails badly and even soft forecasts can underperform SLO-only governance on some traces. The paper therefore treats shape classification as an open calibration problem.

3.4 Policy Cadence

A key engineering constraint is that governance cannot enter the token hot path. YieldOS-Lite computes policy snapshots on a configurable slow-path cadence. The tick sweeps show a workload-dependent knee: 100ms is best in the normal tick sweep, while 75ms is best in stress. Very fast ticks lose to overhead; very slow ticks go stale.

4 Metrics

4.1 Governed Goodput

Governed goodput counts only completed output tokens from requests that satisfy both TTFT and ITL SLOs, then divides by simulated GPU time plus control-plane overhead:

$$\text{GovernedGoodput} = \frac{\sum_{r \in R} \mathbf{1}[r \models TTFT \wedge r \models ITL] \cdot \text{outputTokens}(r)}{\text{simulatedGPUTime} + \text{controlPlaneOverhead}} \tag{2}$$

This metric rejects raw throughput Goodharting. A policy cannot win by producing many tokens that fail latency requirements.

4.2 Obligation Heterogeneity Index

To estimate when governance should help, the simulator adds a coarse OHI:

$$\text{OHI} = \frac{H(PromptBucket) + H(OutputBucket) + H(Priority) + H(SLO) + PrefixReuse + KVPressure}{6} \tag{3}$$

Here $H(\cdot)$ is normalized entropy over the corresponding bucketed field, and $PrefixReuse$ and $KVPressure$ are scaled to $[0, 1]$. OHI is not proposed as a final theoretical metric. It is an MVP diagnostic used to test whether governance advantage tracks workload heterogeneity.

5 Simulator and Reproducibility

The repository implements a dependency-free Phase 1 simulator. It includes synthetic heterogeneous trace generation, replay of CSV/JSONL traces, a workload-suite runner, baselines, YieldOS-Lite policies, ablations, decision logs, and human-readable reports.

Listing 1: Representative simulator commands.

```
pip install -e .
python3 -m yieldos.cli run --requests 800 --seed 7 --out runs/demo
python3 -m yieldos.cli ablate --requests 800 --seed 7 --out runs/ablations
python3 -m yieldos.cli workload-suite --requests 800 --seed 41 --out runs/workload_suite
python3 -m yieldos.cli replay --trace runs/workload_suite/traces/chat_heavy.csv --out runs/
    replay_chat
PYTHONPATH=src python3 -m unittest discover -s tests
```

The trace replay schema accepts `arrival_time_ms`, `prompt_tokens`, `output_tokens`, `tenant_id`, `priority_class`, optional SLO fields, optional `prefix_id`, and optional `abandoned_at_ms`. The simulator is intentionally coarse: it models control-plane choices rather than CUDA execution.

6 Evaluation

6.1 Policies Compared

The simulator compares:

- vLLM-style continuous batching,
- Sarathi-style chunked-prefill scheduling,
- DistServe-style prefill/decode disaggregation,
- an initial YieldOS-Lite policy,
- the ablation-refined YieldOS-Lite policy,
- ablations including no SLO Notary, LRU KV, no classifier, noisy forecasts, policy cadence variants, and others.

These are style-level simulator baselines, not unmodified production implementations of vLLM, Sarathi-Serve, or DistServe. They are intended to test control-plane allocation behavior before integration with a real serving engine.

6.2 MVP Baseline Comparison

The first MVP comparison used 800 requests with seed 7. Table 1 and Figure 2 show the result. YieldOS-Lite achieves 486.41 governed tokens/GPU-second, compared with 401.63 for the DistServe-style baseline.

Table 1: MVP baseline comparison: 800 requests, seed 7.

Policy	Governed goodput	SLO attainment	TTFT p95	ITL p95
YieldOS-Lite	486.41	0.274	30163.0 ms	325.0 ms
DistServe-style	401.63	0.258	32658.0 ms	325.0 ms
Sarathi-style	126.64	0.076	39016.6 ms	905.0 ms
Continuous batching	40.15	0.033	49318.0 ms	625.0 ms

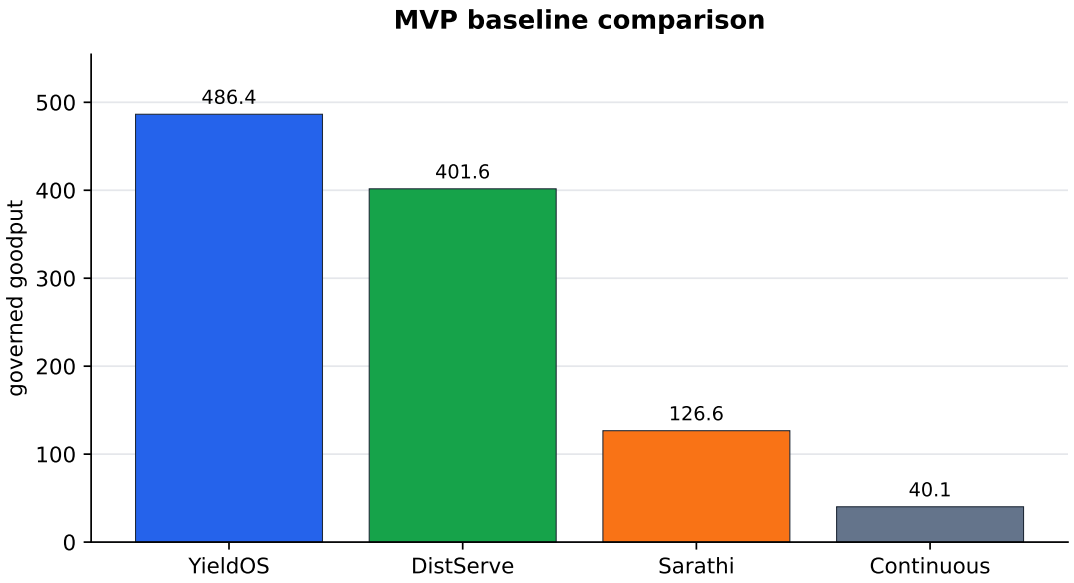

Figure 2: MVP baseline comparison. Governed goodput counts only SLO-valid output tokens and includes control-plane overhead.

6.3 MVP Ablations

The strongest ablation signal was removal of the SLO Notary. Governed goodput dropped from 486.41 to 337.82 tokens/GPU-second. KV Treasury beat LRU on this seed in both governed goodput and recompute waste. A couple of control ablations, including `no_fallback_lane` and `no_online_reclass`, slightly exceeded the initial policy, motivating the ablation-refined policy used for the main result.

Table 2: Selected MVP ablations from the initial run.

Policy	Governed goodput	SLO attainment	TTFT p95	ITL p95
ablate_no_fallback_lane	495.34	0.281	30852.4 ms	300.0 ms
ablate_no_online_reclass	493.61	0.282	31086.0 ms	500.0 ms
ablate_no_shape_classifier	489.73	0.287	28878.5 ms	175.0 ms
Initial YieldOS-Lite policy	486.41	0.274	30163.0 ms	325.0 ms
ablate_lru_kv	481.92	0.282	31115.2 ms	400.0 ms
ablate_noisy_forecasts	473.53	0.282	31237.2 ms	400.0 ms
ablate_policy_tick_1ms	465.89	0.278	31053.8 ms	450.0 ms
ablate_policy_tick_100ms	460.49	0.269	30535.8 ms	375.0 ms
ablate_no_slo_notary	337.82	0.200	30350.9 ms	250.0 ms

The ablation changed the architecture: shape forecasts became soft evidence rather than hard routing, fallback demotion was removed, and online reclassification became hysteretic.

6.4 YieldOS-Lite Main Result

Table 3 and Figure 3 show the main YieldOS-Lite comparison after ablation-guided refinement. YieldOS-Lite reaches 507.88 governed tokens/GPU-second, improving over the initial policy by 4.4% and over DistServe-style disaggregation by 26.5%.

Table 3: YieldOS-Lite default comparison after ablation-guided refinement.

Policy	Goodput	SLO	TTFT p95	ITL p95	KV hits	KV recompute waste
YieldOS-Lite	507.88	0.286	28960.4 ms	203.7 ms	65176	296950.1
Initial YieldOS-Lite policy	486.41	0.274	30163.0 ms	325.0 ms	73765	319145.8
DistServe-style	401.63	0.258	32658.0 ms	325.0 ms	64488	389913.4
Sarathi-style	126.64	0.076	39016.6 ms	905.0 ms	72184	378287.7
Continuous batching	40.15	0.033	49318.0 ms	625.0 ms	56268	413997.3

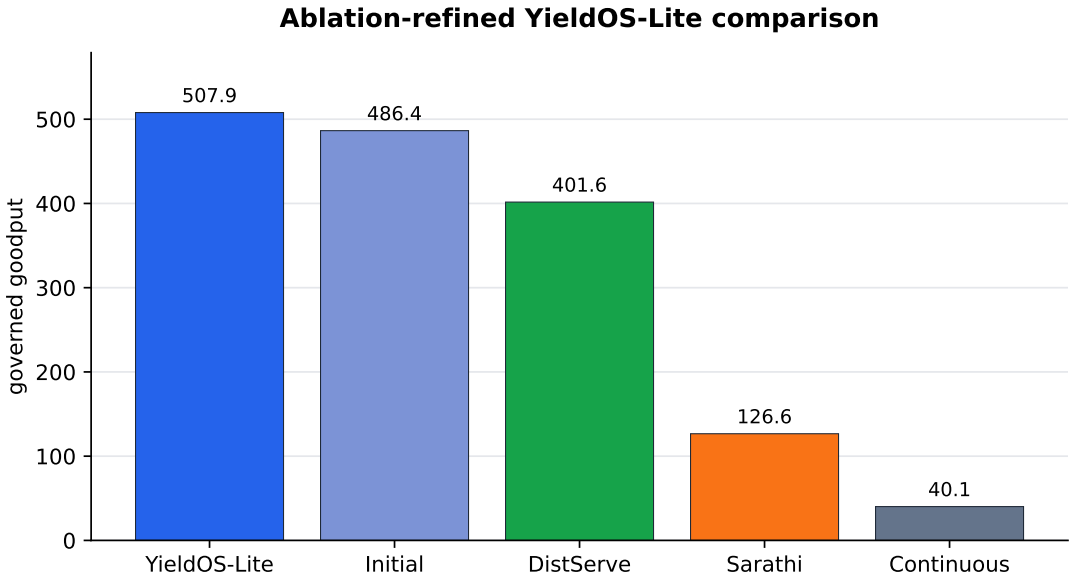

Figure 3: YieldOS-Lite improves governed goodput over the initial policy and all simulated baselines in the default comparison.

Compared with the initial policy, the ablation-refined YieldOS-Lite policy improves TTFT p95 from 30163.0 ms to 28960.4 ms, improves ITL p95 from 325.0 ms to 203.7 ms, reduces overhead from 3029.5 ms to 2938.5 ms, and reduces KV recompute waste from 319145.8 to 296950.1. This suggests that the refined policy is not merely increasing throughput by violating latency; it improves the governed metric and its supporting latency indicators.

6.5 Severe KV-Pressure Stress

The KV-heavy stress run is a severe-contention regime. Absolute SLO attainment is low across all policies, but YieldOS-Lite still wins on governed goodput. Table 4 shows the result.

Table 4: KV-heavy stress run. The regime is collapse-level: low SLO attainment indicates that no policy has enough capacity to fully satisfy demand.

Policy	Goodput	SLO	TTFT p95	ITL p95	KV hits	Recompute waste	Overhead
YieldOS-Lite	29.82	0.046	74495.0 ms	25.0 ms	193728	5071964.8	5231.7
YieldOS-Lite with LRU KV	28.12	0.044	73831.5 ms	25.0 ms	219256	5223037.7	5226.6
YieldOS-Lite without SLO Notary	27.37	0.034	76619.5 ms	25.0 ms	172026	5088429.8	5648.3
DistServe-style	18.30	0.014	85286.1 ms	25.0 ms	176084	4877996.1	5717.9

This run should be read as degradation management, not as a claim that scheduling can overcome insufficient capacity. The interesting result is that YieldOS-Lite preserves more SLO-valid work than the baselines under collapse-level contention.

6.6 Load Sweep

Across tested load regimes, YieldOS-Lite beats DistServe-style disaggregation at every point. Figure 4 shows the gain over DistServe.

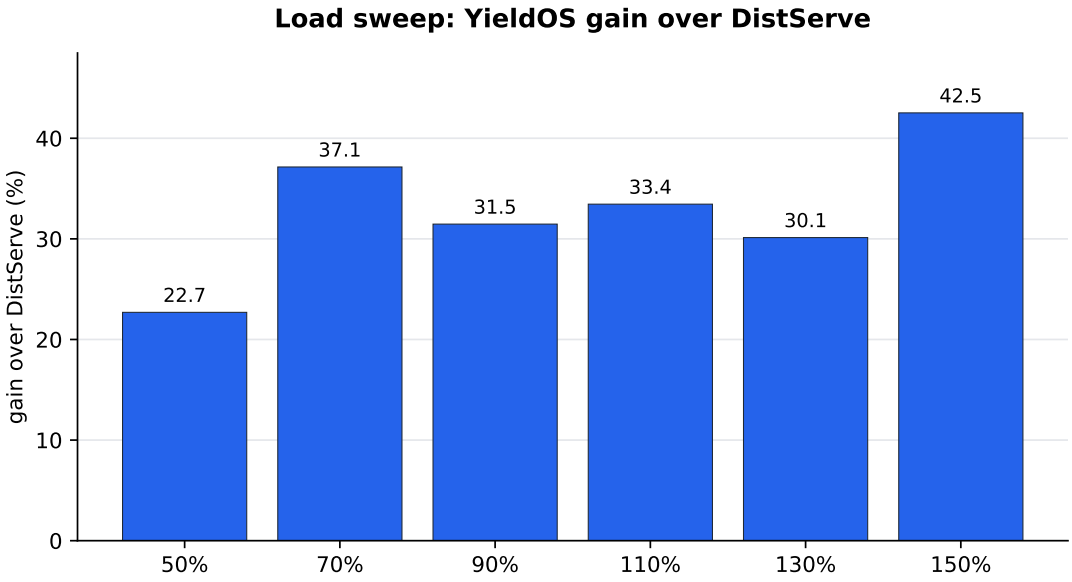

Figure 4: Load sweep: YieldOS-Lite gain over DistServe-style disaggregation. Gains remain positive from 50% to 150% load.

Table 5: Load sweep gain over DistServe-style baseline.

Offered load	YieldOS-Lite gain over DistServe-style
50%	+22.7%
70%	+37.1%
90%	+31.5%
110%	+33.4%
130%	+30.1%
150%	+42.5%

6.7 SLO Tightness Sweep

The SLO tightness sweep isolates SLO pressure as the experimental variable. Removing the SLO Notary hurts under all tested tightness levels, as shown in Figure 5.

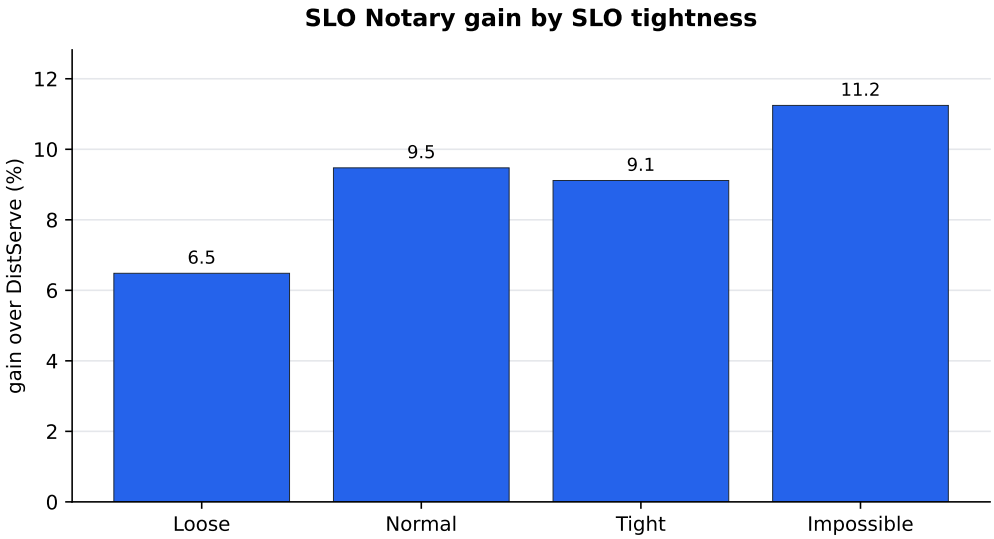

Figure 5: SLO Notary gain over no-Notary variants under the SLO tightness sweep.

Table 6: SLO Notary benefit by SLO tightness.

SLO tightness	Gain over no-Notary
Loose	+6.5%
Normal	+9.5%
Tight	+9.1%
Impossible	+11.2%

The load sweep complicates the story slightly because no-Notary remains competitive at some individual load points. The more precise claim is that SLO Notary is most useful when SLO pressure, not merely aggregate load, is the binding constraint.

6.8 Policy Cadence Sweep

The normal tick sweep finds 100ms as the best cadence at 522.12 governed tokens/GPU-second; 75ms and 50ms are close. 5–10ms loses to overhead, while 200ms goes stale. Under stress, the best cadence shifts to 75ms at 32.59 governed tokens/GPU-second. This validates the slow-path/hot-path design: policy should be frequent enough to avoid stale governance, but not so frequent that governance overhead starves the dispatch path.

6.9 Semi-Realistic Workload Profiles

The workload-suite runner produces production-shaped profiles: chat-heavy, RAG-heavy, code-heavy, batch-summary-heavy, and mixed-enterprise. Results are shown in Table 7 and Figure 6.

Table 7: Semi-realistic workload profiles: YieldOS-Lite vs DistServe-style.

Profile	YieldOS-Lite	DistServe-style	Improvement
chat-heavy	530.69	488.50	+8.6%
RAG-heavy	41.10	27.83	+47.6%
code-heavy	286.30	209.77	+36.5%
batch-summary-heavy	85.85	51.13	+67.9%
mixed-enterprise	172.82	77.90	+121.8%

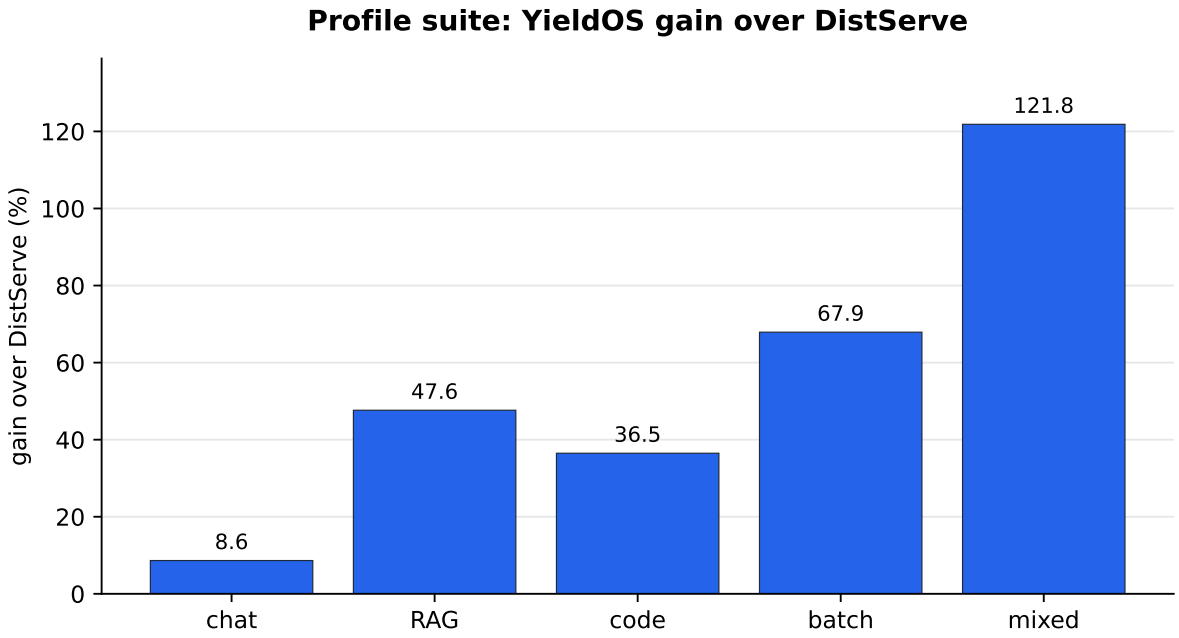

Figure 6: Profile gains over DistServe-style disaggregation. Gains are smallest in chat-heavy traffic and largest in mixed-enterprise traffic.

The pattern is consistent with the hypothesis: the more heterogeneous the obligations, the larger the governance advantage.

6.10 External Trace Pilot: BurstGPT Boundary Condition

BurstGPT provides a public real-world trace dataset for LLM serving workloads. The public repository states that it contains real-world LLM serving traces with request timestamps, model identifiers, request-token counts, response-token counts, total-token counts, and log type [6]. The YieldOS-Lite adapter normalizes BurstGPT fields into the replay schema. We avoid treating exact release-size counts as part of the experimental claim because the public dataset has multiple release files and versions.

The pilot used the first 800 rows of the public BurstGPT trace. This sample is almost entirely interactive, has no prefix-reuse signal, and is relatively homogeneous compared with the mixed-enterprise profiles. Table 8 and Figure 7 show the boundary condition.

Table 8: BurstGPT pilot. The external sample is a homogeneous interactive boundary condition where DistServe-style disaggregation is competitive or better.

Time scale	Best policy	Note
20x	DistServe-style, tied on goodput	very light contention; all goodput values effectively equal
50x	DistServe-style, tied on goodput	still mostly equal
100x	DistServe-style	DistServe 507.77 vs YieldOS-Lite 491.15
200x	DistServe-style	DistServe 442.61 vs YieldOS-Lite 367.68

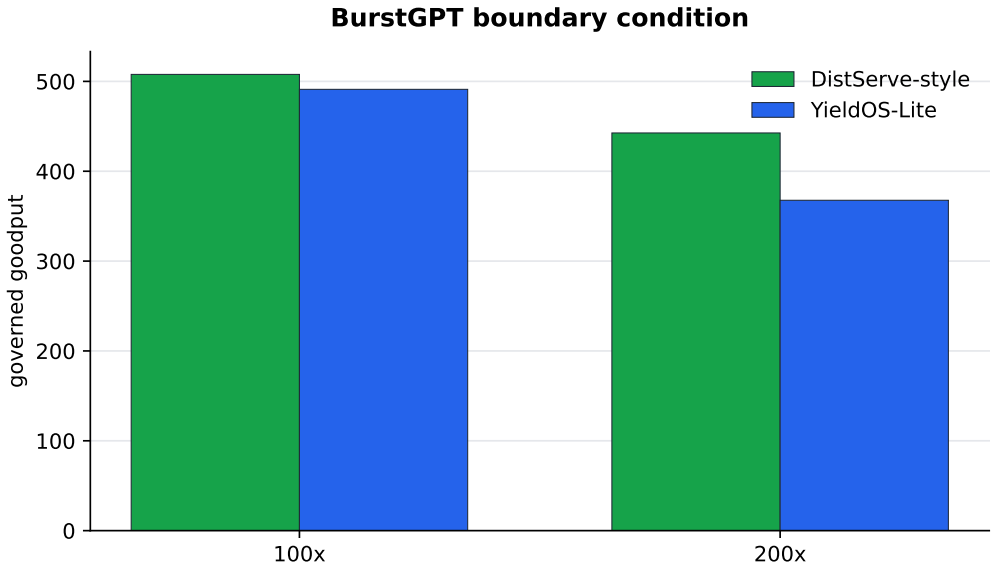

Figure 7: BurstGPT pilot at 100x and 200x. In a mostly interactive, low-prefix-reuse external sample, DistServe-style disaggregation can outperform YieldOS-Lite.

This pilot is deliberately not folded into the main positive claim. It clarifies the operating domain: YieldOS-Lite helps most when workload heterogeneity, prefix reuse, tenant priority, and SLO pressure create competing obligations. When requests are homogeneous and interactive, mechanistic disaggregation may be better.

6.11 Obligation Heterogeneity Analysis

Table 9 and Figure 8 show the OHI analysis. Across the small pilot set, OHI and YieldOS gain have a Pearson correlation of 0.713.

Table 9: Obligation Heterogeneity Index and YieldOS gain over DistServe-style baseline.

Workload	OHI	YieldOS gain over DistServe-style
BurstGPT sample, 100x	0.315	-3.3%
chat-heavy	0.540	+8.6%
code-heavy	0.666	+36.5%
mixed-enterprise	0.711	+121.8%
batch-summary-heavy	0.722	+67.9%
RAG-heavy	0.763	+47.6%

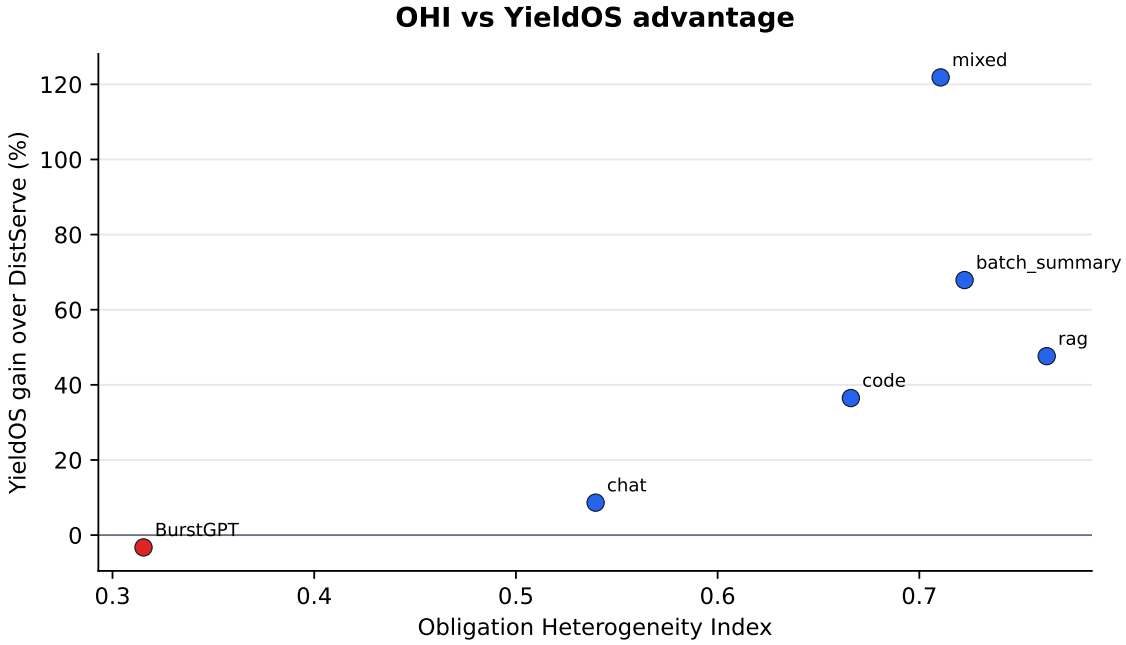

Figure 8: OHI vs. YieldOS advantage. The pilot correlation is suggestive, not definitive. The hypothesis is that governance advantage grows with obligation heterogeneity.

The current scientific hypothesis is:

> Governance advantage increases when requests impose heterogeneous obligations on prefill compute, decode bandwidth, KV residency, SLO slack, tenant priority, and prefix reuse.

7 Lessons

Lesson 1: SLO pressure is not the same as load. The load sweep shows positive gains across load regimes, but the SLO tightness sweep isolates where the SLO Notary is most useful: when SLO pressure is the binding constraint.

Lesson 2: KV hit rate is not KV value. LRU can produce more raw KV hits while preserving lower-value KV. Value-aware KV governance should be evaluated by value preserved, not hit rate alone.

Lesson 3: Shape classification should not become routing authority too early. The shape-stress result falsifies hard routing. Shape forecasts should remain advisory until calibration improves.

Lesson 4: Governance cadence has a knee. Very frequent policy updates lose to overhead; slow updates go stale. The slow-path control plane must publish policy snapshots at a workload-sensitive cadence.

Lesson 5: Boundary conditions are part of the result. The BurstGPT pilot strengthens the paper because it shows where YieldOS-Lite does not win. Homogeneous interactive traces may not require rich resource governance.

8 Future Work: From YieldOS-Lite to YieldOS Marketplace

YieldOS-Lite is Phase 1. The full YieldOS vision is a marketplace of resource obligations.

8.1 Resource Obligation Graphs

Future versions should decompose each request into explicit obligations over prefill FLOPs, decode slots, KV residency, transfer cost, speculation, SLO slack, and tenant priority. Scheduling then becomes optimization over obligations rather than over whole requests.

8.2 Speculation Market

Speculative decoding should be governed by expected yield: draft cost, verification cost, acceptance probability, latency gain, and opportunity cost. Speculation should be admitted only when expected governed-goodput gain is positive.

8.3 Idle Slot Futures

Future systems can convert predicted idle GPU windows into preemptible work slots for checkpointable or disposable tasks: offline embeddings, KV compression, cache warming, draft generation, or evaluation probes. These tasks must be revocable and must not harm foreground SLOs.

8.4 Production Engine Integration

The next milestone is not to replace vLLM or TensorRT-LLM. It is to integrate YieldOS as a sidecar control plane: collect metrics, publish policy snapshots, influence admission and prioritization, and measure governed goodput on production-like traces.

9 Limitations

The objective of this paper is to evaluate whether LLM inference resource governance is a promising control-plane direction. It is not to demonstrate a production-ready serving engine. The artifact is a coarse trace simulator, so the results should be read as evidence about policy behavior, not as CUDA-level speedup, direct vLLM/TensorRT-LLM replacement, or measured production GPU utilization improvement.

Several limitations follow from that scope. Synthetic and semi-realistic traces are useful for controlled policy exploration, but they are not final systems validation. The BurstGPT pilot is small and limited to the first 800 rows. OHI is an MVP diagnostic for explaining observed gains, not a final workload theory. Shape classification is not validated as hard routing and should remain advisory. KV Treasury requires more realistic transfer, eviction, and recompute-cost modeling. Control-plane overhead is simulated and must be measured inside a real serving engine.

The intended reader takeaway is therefore specific: YieldOS-Lite can be tried today as a reproducible simulator and trace-replay scaffold, but not as a drop-in production scheduler. The current evidence justifies the next step: integrate the control plane with a real inference engine or production-like replay environment and test whether the same governance advantage survives real execution costs.

10 Conclusion

YieldOS-Lite reframes LLM inference efficiency as a resource-governance problem. Existing systems provide powerful mechanisms: continuous batching, KV paging, chunked prefill, and prefill/decode disaggregation. This paper asks whether those mechanisms need a higher-level control plane when workloads contain heterogeneous obligations over compute, memory, latency slack, priority, and prefix reuse.

The answer from this Phase 1 simulator is cautiously positive. YieldOS-Lite improves governed goodput in synthetic heterogeneous workloads and semi-realistic profile suites, while the BurstGPT pilot shows an important boundary condition: when traffic is homogeneous, interactive, and low in prefix reuse, mechanistic disaggregation can be competitive or better. This makes the result narrower and more useful.

YieldOS-Lite is not a universal replacement for existing serving engines; it is evidence that governance becomes valuable when scheduling alone cannot resolve competing obligations.

The practical takeaway is also constrained. Researchers and engineers can run YieldOS-Lite today to reproduce the simulator results, replay traces, inspect policy decisions, and test whether their workloads have enough obligation heterogeneity to justify governance. The next milestone is production-like validation: integrate YieldOS as a sidecar control plane over an existing inference engine, measure real overheads, and test governed goodput on larger external or private traces. If that step preserves the simulator ordering, resource governance becomes a credible next layer of LLM inference efficiency.

References

[1] Gyeong-In Yu et al. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. OSDI 2022. https://www.usenix.org/conference/osdi22/presentation/yu

[2] vLLM Project. *vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention*. 2023. https://vllm.ai/blog/2023-06-20-vllm

[3] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. https://arxiv.org/abs/2309.06180

[4] Amey Agrawal et al. *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve*. OSDI 2024. https://www.usenix.org/conference/osdi24/presentation/agrawal

[5] Yinmin Zhong et al. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[6] HPMLL. *BurstGPT: A ChatGPT/GPT-3.5 and GPT-4 Workload Trace to Optimize LLM Serving Systems*. GitHub repository. https://github.com/HPMLL/BurstGPT

[7] Y. Wang et al. *BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems*. KDD 2025 / public paper listing. https://huggingface.co/papers/2401.17644